

# Multimodal visual understand+SLAM navigation(Text Version)

---

## Multimodal visual understand+SLAM navigation(Text Version)

1. Course Content
2. Preparation
  - 2.1 Content Description
  - 2.2 Configuring the Mapping File
  - 2.3 Configuring Map Mapping Variables in Dify
3. Run Case
  - 3.1 Starting the Program
    - 3.1.1 Jetson Nano board Startup Steps:
    - 3.1.1 RDKX5, Raspberry Pi PI5 and ORIN board startup steps:
  - 3.2 Test Cases
    - 3.2.1 Example 1
4. Source Code Parsing
  - 4.1 Example 1

## 1. Course Content

---

1. Learn to use the robot's visual understanding combined with SLAM navigation.
2. Learn new key source code.

## 2. Preparation

---

### 2.1 Content Description

This course uses the Raspberry Pi 5 as an example. For Raspberry Pi and Jetson Nano boards, you need to open a terminal on the host computer and enter the command to enter the Docker container. Once inside the Docker container, enter the commands mentioned in this course in the terminal. For instructions on entering the Docker container from the host computer, refer to **[01. Robot Configuration and Operation Guide] -- [5.Enter Docker (For JETSON Nano and RPi 5)]**. For RDKX5 and Orin boards, simply open a terminal and enter the commands mentioned in this course.

 This example uses `model:"qwen/qwen2.5-v1-72b-instruct:free", "qwen-v1-latest"`

 The responses from the big model for the same test command may not be exactly the same and may differ slightly from the screenshots in the tutorial. To increase or decrease the diversity of the big model's responses, refer to the section on configuring the decision-making big model parameters in **[04.AI Large Model Development] -- [5.Deploy the RAG knowledge base]** .

### 2.2 Configuring the Mapping File

For detailed information on the principles and concepts of map mapping, please refer to the course content in [AI Model Basics - 3. Embodied Intelligent Robot System Architecture].

**For Raspberry Pi 5 and Jetson Nano controllers, you must first enter the Docker container. For RDKX5 and Orin board, this is not necessary.**

Use VNC to connect to the robot desktop. Open a terminal in Docker and enter the command.

```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```

Create a new terminal on the virtual machine and start the command.\*\* (For Raspberry Pi and Jetson Nano, it is recommended to run visualization in the virtual machine.)

```
ros2 launch yahboomcar_nav display_nav_launch.py
```

Wait for the navigation algorithm to start to display the map. Then open the Docker terminal and enter the command.

```
#Choose one of the two navigation algorithms
#Normal navigation
ros2 launch yahboomcar_nav navigation_teb_launch.py

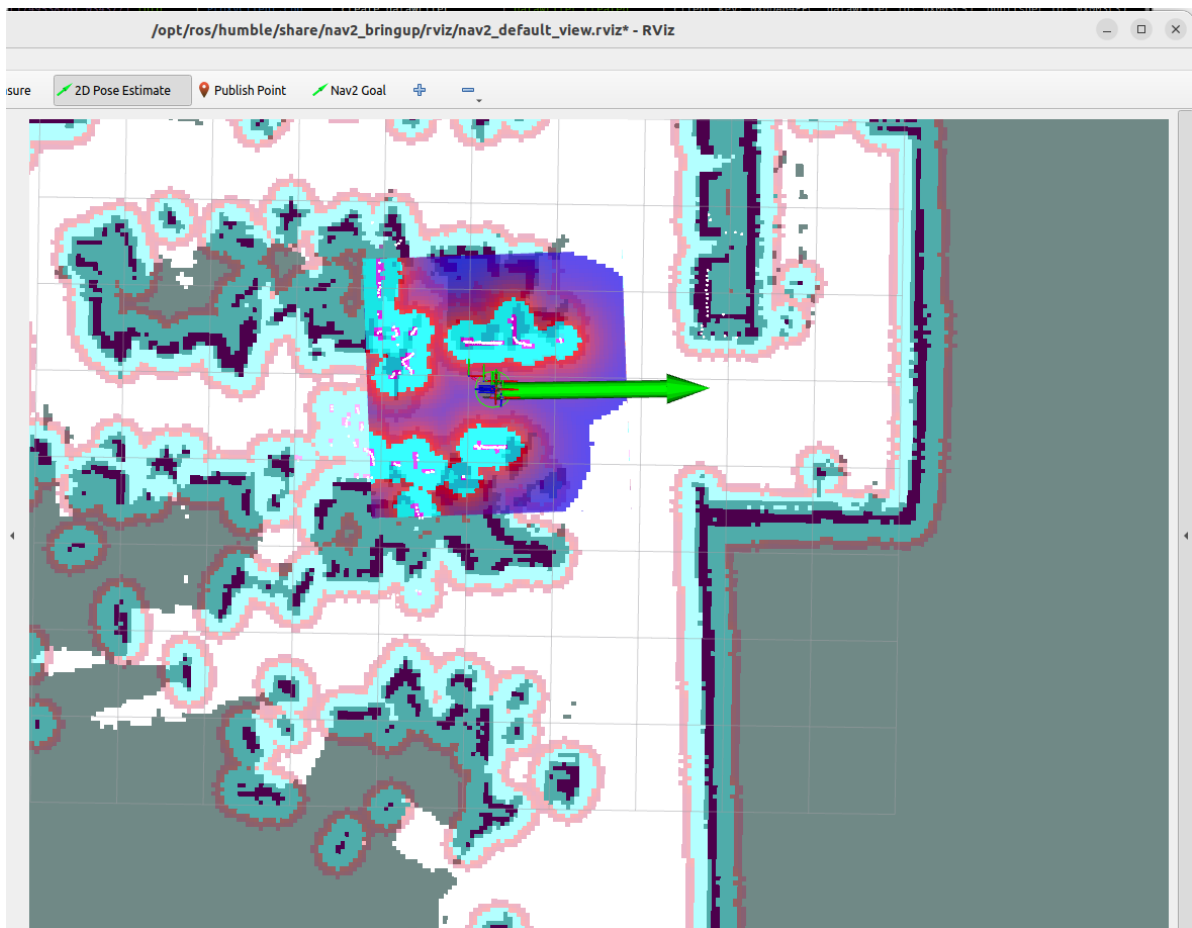
#Fast repositioning navigation (Not supported on Raspberry Pi 5 or Jetson Nano
controllers)
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomca
r_nav/maps/yahboomcar.pbstream

ros2 launch yahboomcar_nav navigation_cartodwb_launch.py
maps:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahb
oomcar.yaml
```

**Note: yahboomcar.yaml and yahboomcar.pbstream must be mapped simultaneously, meaning they are the same map. Refer to the cartograph mapping algorithm to save the map.**

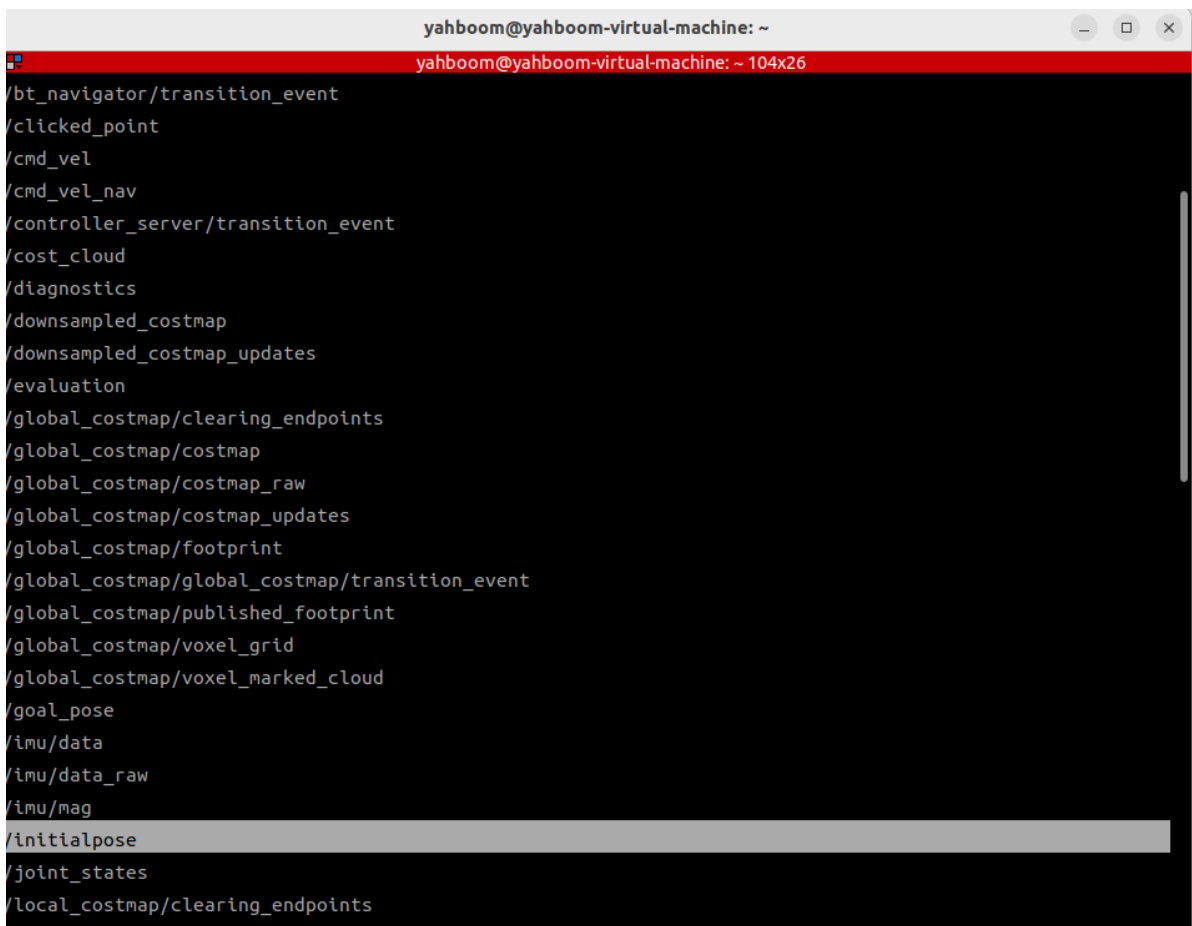
The map visualization interface will appear in rviz2. Click **2D Pose Estimate** in the upper toolbar to select it. Roughly mark the robot's position and orientation on the map.

The robot model will appear on the map as shown below:



Open a new terminal in the virtual machine and enter the command

```
ros2 topic list
```

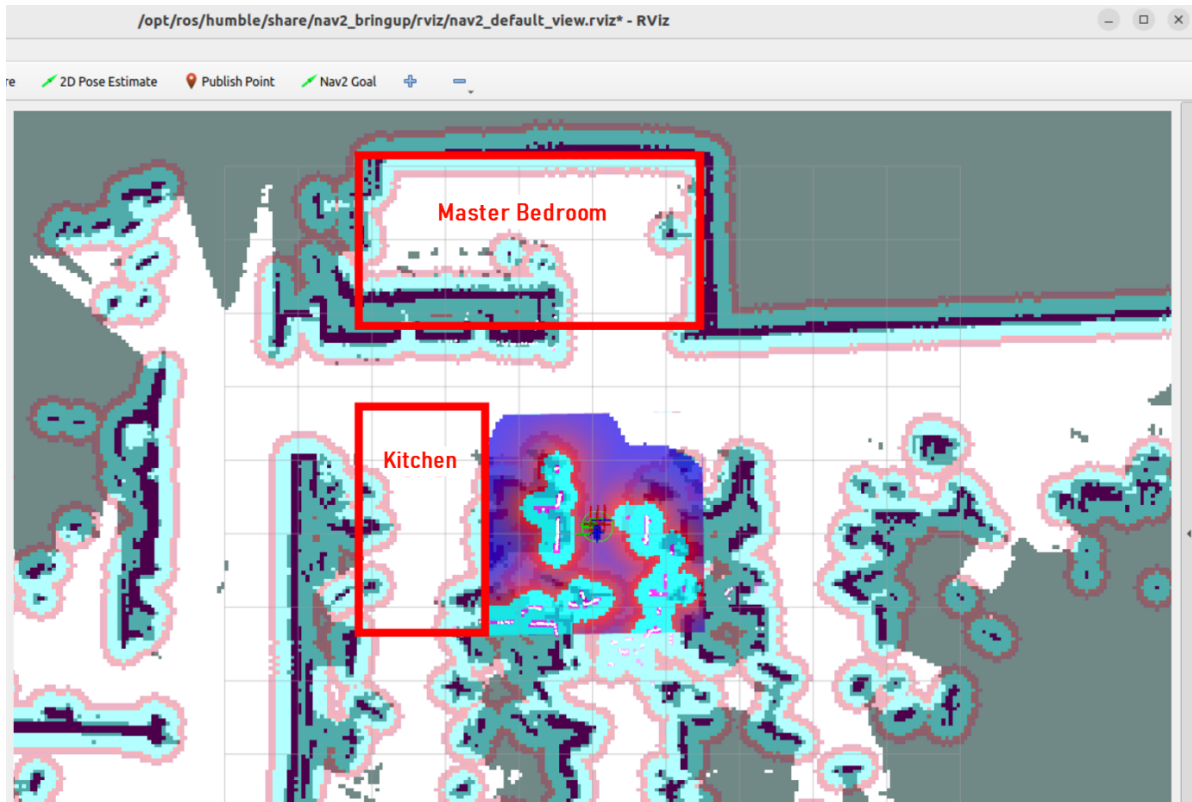


You can see the **/initialpose** topic. This topic is published by the **2D Pose Estimate** tool in rviz2 in the previous step. We can use this tool to publish coordinates to view the data of the **/initialpose** topic and obtain the coordinates and orientation angle of a certain point in the map.

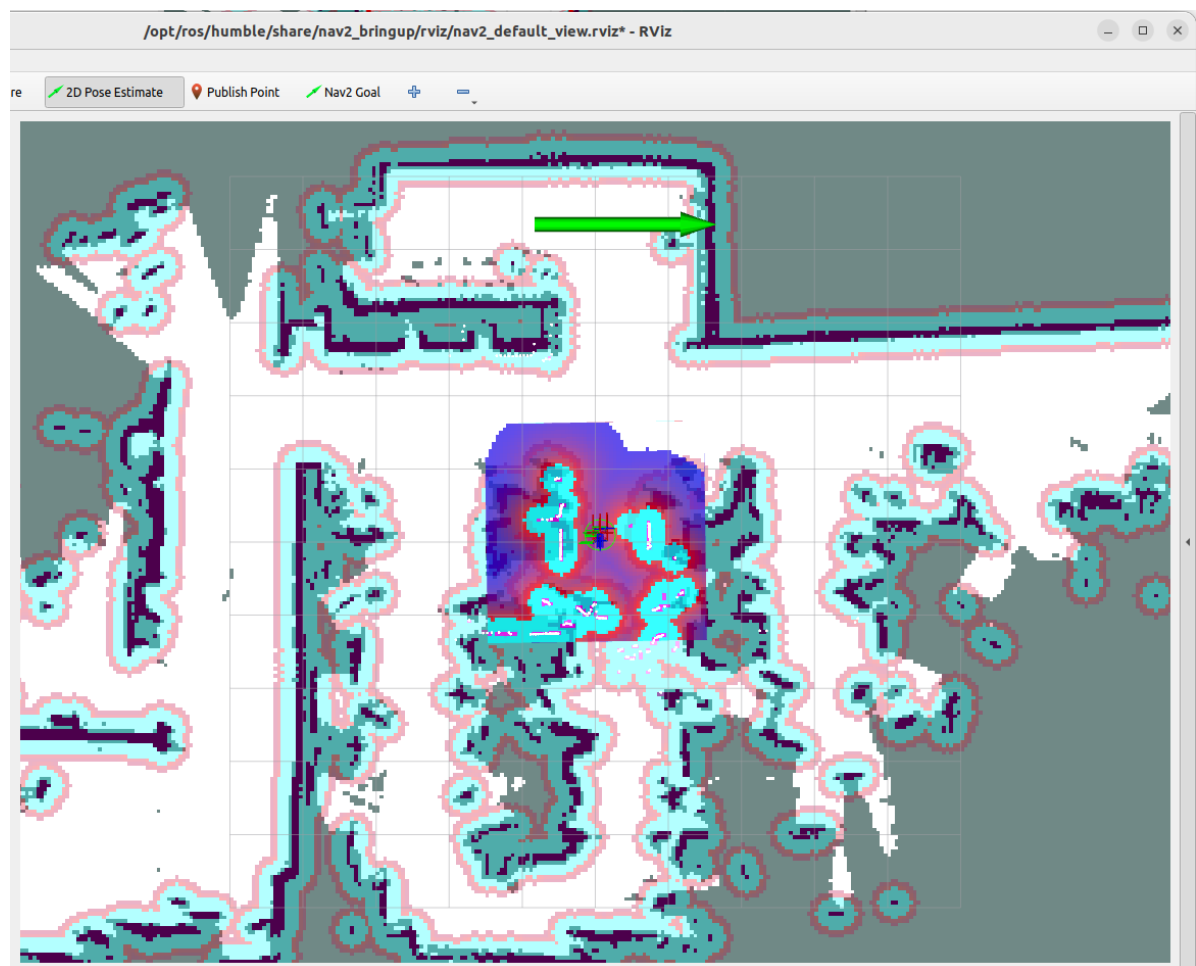
Enter the following command in the terminal to observe the data on the **/initialpose** topic.

```
ros2 topic echo /initialpose
```

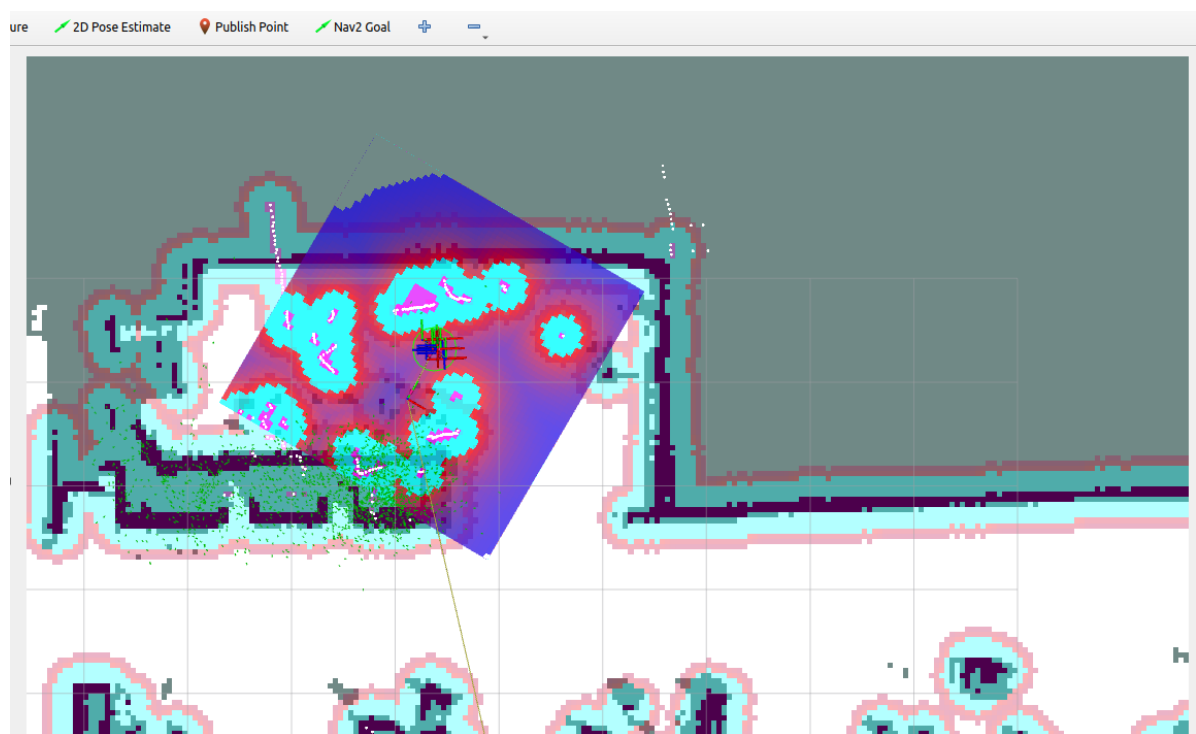
We can name any point in the map. Here we take "Master Bedroom" and "Kitchen" as examples.



As shown in the figure below, we first click **2D Pose Estimate** tool, then select a place in the "Master Bedroom" area and click the left mouse button and hold it to adjust the direction. After confirming, release it.



The robot position will be adjusted to the position and direction just selected. The robot's estimated position and direction can be previewed in rviz.



In the last frame of the terminal window, you can see the coordinates of the target point we just marked with the **2D Pose Estimate** tool.



```
x: 0.0
y: 0.0
z: 0.4503881236302555
w: 0.8928328724306793
```

#此处可新增添加区域对应的栅格地图坐标点，注意和上面格式保持一致

#Here, you can add the raster map coordinate points corresponding to the added area. Please note that the format should be consistent with the above

In the same way, we can add a map of the kitchen

#根据实际的场景环境，自定义地图中的区域，可以添加任意个区域，注意和大模型的地图映射保持一致即可  
#According to the actual scene environment, customize the areas in the map. You can add any number of areas, just make sure they are consistent with the map mapping of the large model

#地图映射Map mapping

A:

```
name: 'Master Bedroom'
position:
  x: 1.6878514289855957
  y: -0.32615160942077637
  z: 0.0
orientation:
  x: 0.0
  y: 0.0
  z: 0.4503881236302555
  w: 0.8928328724306793
```

G:

```
name: 'Kitchen'
position:
  x: 2.367471694946289
  y: -1.568994164466858
  z: 0.0
orientation:
  x: 0.0
  y: 0.0
  z: -0.3106425066197889
  w: 0.950526818706855
```

#此处可新增添加区域对应的栅格地图坐标点，注意和上面格式保持一致

#Here, you can add the raster map coordinate points corresponding to the added area. Please note that the format should be consistent with the above

Then switch to the `/yahboomcar_ros2_ws/yahboomcar_ws#` workspace in the terminal:

Recompile the largemodel function package to take effect (**jetson nano, Raspberry Pi host, you need to enter docker first**)

```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws/
```

Recompile the feature package:

```
colcon build --packages-select multi_brains
```

```
sunrise@ubuntu:~/yahboomcar_ros2_ws/yahboomcar_ws$ colcon build --packages-select multi_brains
Starting >>> multi_brains
Finished <<< multi_brains [10.6s]
Summary: 1 package finished [12.2s]
```

## 2.3 Configuring Map Mapping Variables in Dify

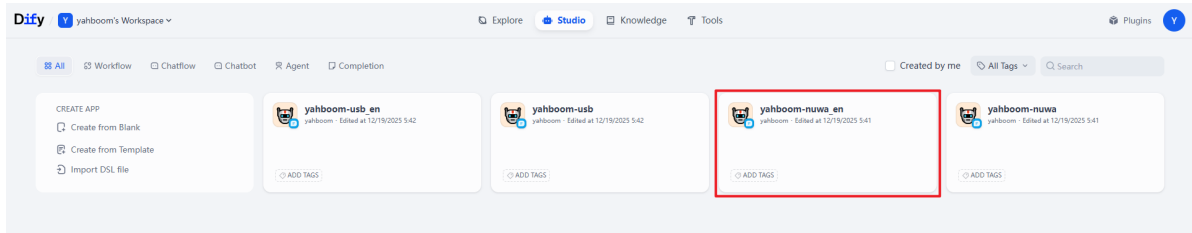
Open the host terminal and start the Dify environment:

```
sh ~/bringup_dify.sh
```

```
pi@raspberrypi: ~
File Edit Tabs Help
pi@raspberrypi1:~ $ sh ~/bringup_dify.sh
[+] Running 13/13
✓ Network docker_ssrf_proxy_network Created 0.1s
✓ Network docker_default Created 0.1s
✓ Container docker-weaviate-1 Started 2.2s
✓ Container docker-redis-1 Started 2.6s
✓ Container docker-web-1 Started 3.1s
✓ Container docker-sandbox-1 Started 2.2s
✓ Container docker-db-1 Healthy 4.6s
✓ Container docker_ssrf_proxy-1 Started 3.1s
✓ Container docker-worker_beat-1 Started 7.3s
✓ Container docker-worker-1 Started 7.2s
✓ Container docker-plugin_daemon-1 Started 6.2s
✓ Container docker-api-1 Started 6.8s
✓ Container docker-nginx-1 Started 7.7s
pi@raspberrypi1:~ $
```

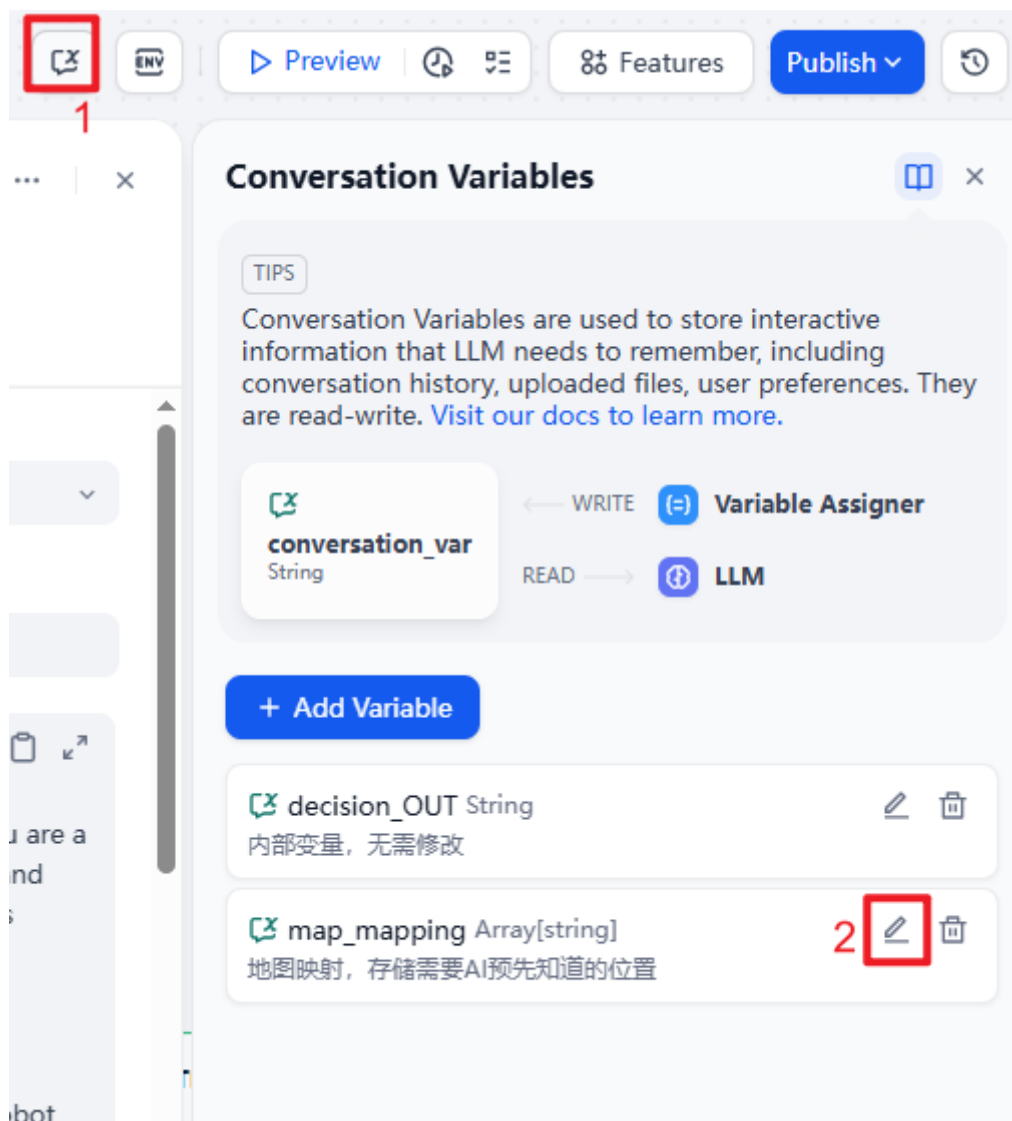
- Enter the vehicle's IP address directly into the browser's address bar to access the Dify management page, then click to select the corresponding AI application.
- International users: yahboom\_en

Example:

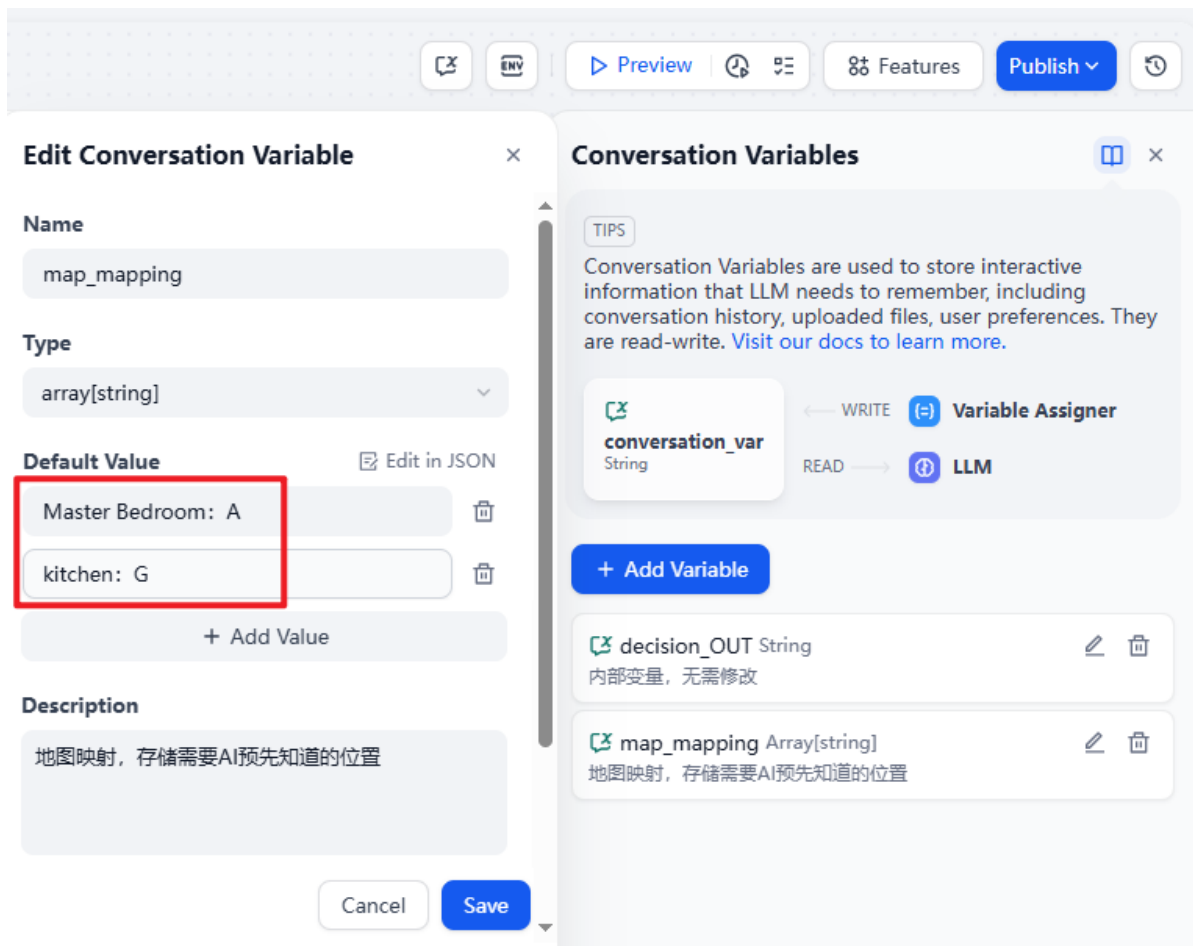


- Click on `session variables` in the upper right corner, then click the edit button for the `map_mapping` variable.

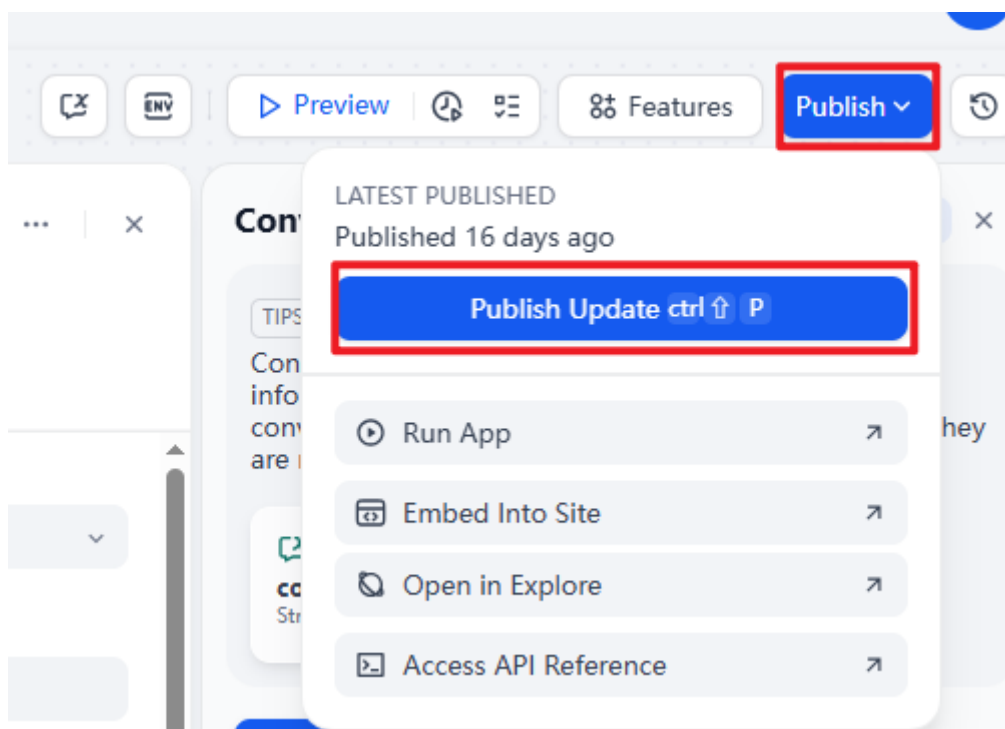




- In the pop-up `Edit Session variables` window, edit the mapping relationship according to the settings in the previous map mapping file, and then click Save.



- Finally, remember to click Publish -> Publish Update to save your changes.



### 3. Run Case

## 3.1 Starting the Program

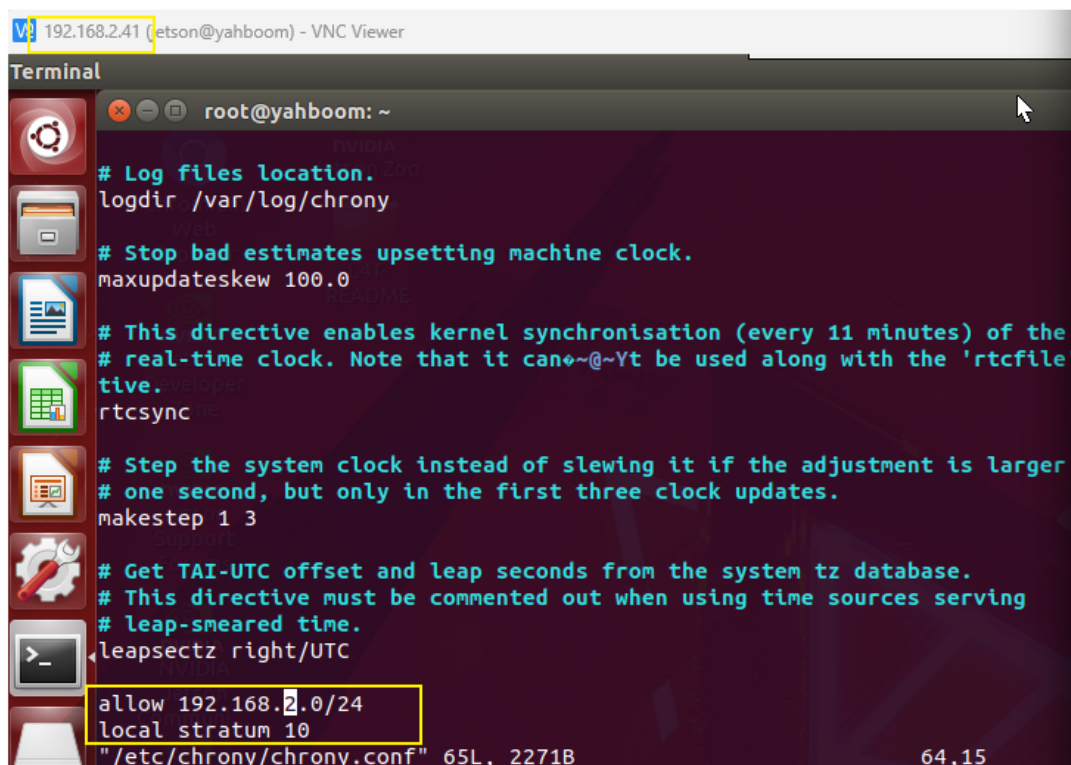
### 3.1.1 Jetson Nano board Startup Steps:

Due to Nano performance issues, navigation-related nodes must be run on a virtual machine. Therefore, navigation performance is highly dependent on the network. We recommend running in an indoor environment with a good network connection. The following configuration is required:

- **Board Side (Requires Docker)**

Open a terminal and enter

```
sudo vi /etc/chrony/chrony.conf
```



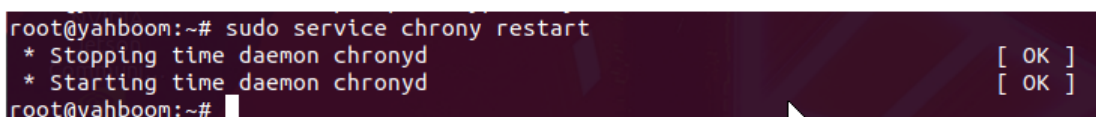
```
192.168.2.41 (jetson@yahboom) - VNC Viewer
Terminal
root@yahboom: ~
# Log files location.
logdir /var/log/chrony
# Stop bad estimates upsetting machine clock.
maxupdateskew 100.0
# This directive enables kernel synchronisation (every 11 minutes) of the
# real-time clock. Note that it can't be used along with the 'rtcfiler'
# directive.
rtcsync
# Step the system clock instead of slewing it if the adjustment is larger
# one second, but only in the first three clock updates.
makestep 1 3
# Get TAI-UTC offset and leap seconds from the system tz database.
# This directive must be commented out when using time sources serving
# leap-smeared time.
leapsectz right/UTC
allow 192.168.2.0/24
local stratum 10
"/etc/chrony/chrony.conf" 65L, 2271B 64,15
```

Add the following two lines at the end of the file (192.168.2.0/24 is filled in according to the actual network segment. Here we take the current board IP address 192.168.2.41 as an example)

```
allow 192.168.x.0/24    #x represents the corresponding network segment
local stratum 10
```

After saving and exiting, enter the following command to take effect:

```
sudo service chrony restart
```



```
root@yahboom:~# sudo service chrony restart
* Stopping time daemon chronyd [ OK ]
* Starting time daemon chronyd [ OK ]
root@yahboom:~#
```

- **VM side**

Open a terminal and type

```
echo "server 192.168.2.41 iburst" | sudo tee
/etc/chrony/sources.d/jetson.sources
```

The 192.168.2.41 entry above is the board's IP address.

Enter the following command to take effect.

```
sudo chronyc reload sources
```

Then enter the following command to check the delay. If the IP address appears, it means it is normal.

```
chronyc sources -v
```

```
yahboom@VM:~$ chronyc sources -v
.-- Source mode  '^' = server, '=' = peer, '#' = local clock.
/  -- Source state '*' = current best, '+' = combined, '-' = not combined,
| /      'x' = may be in error, '~' = too variable, '?' = unusable.
||
||      Reachability register (octal) --.      .- xxxx [ yyyy ] +/- zzzz
||      Log2(Polling interval) --.      |      xxxx = adjusted offset,
||                                  |      yyyy = measured offset,
||                                  |      zzzz = estimated error.
||                                  \      /
||                                  |      |
MS Name/IP address             Stratum Poll Reach LastRx Last sample
=====
^- prod-ntp-5.ntp1.ps5.cano>    2   7   277   133  -4048us[-5395us] +/- 132ms
^- alphyn.canonical.com        2   8   177    66  -193us[-193us] +/- 134ms
^- prod-ntp-3.ntp1.ps5.cano>    2   8   367   131   -16ms[-18ms] +/- 127ms
^- prod-ntp-4.ntp4.ps5.cano>    2   8   377   129 -4907us[-6254us] +/- 133ms
^* time.nju.edu.cn             1   7   377    69   +77us[-1270us] +/- 17ms
^+ 139.199.215.251             2   8   377   135 +2358us[+1011us] +/- 46ms
^+ 119.28.183.184              2   7   377   193 +2061us[+713us] +/- 28ms
^+ 119.28.206.193              2   8   367    8  +2058us[+2058us] +/- 36ms
^+ 192.168.2.41                4   6   377    6   -12ms[-12ms] +/- 92ms
yahboom@VM:~$
```

- **Start the program**

Open the host machine terminal and start the Dify environment:

```
sh ~/bringup_dify.sh
```

Open the terminal in Docker and enter the following command:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

After initialization, the following content will be displayed:

```
[usb_cam_node_exe-1] YUYV 4:2:2 800 x 600 (20 Hz)
[usb_cam_node_exe-1] YUYV 4:2:2 1280 x 1024 (6 Hz)
[usb_cam_node_exe-1] YUYV 4:2:2 320 x 240 (30 Hz)
[usb_cam_node_exe-1] [INFO] [1765965894.552487562] [usb_cam]: Setting 'white_balance_temperature_auto' to 1
[usb_cam_node_exe-1] [INFO] [1765965894.552563191] [usb_cam]: Setting 'exposure_auto' to 1
[usb_cam_node_exe-1] [INFO] [1765965894.552578635] [usb_cam]: Setting 'exposure' to 150
[usb_cam_node_exe-1] [INFO] [1765965894.585467996] [usb_cam]: Setting 'focus_auto' to 0
[usb_cam_node_exe-1] [INFO] [1765965894.673818636] [usb_cam]: Timer triggering every 8 ms
[INFO] [model_service-12]: process started with pid [439]
[INFO] [action_service_usb-13]: process started with pid [441]
[action_service_usb-13] [INFO] [1765965908.031574888] [action_service_usb]: ROS_Action_Service Initialization completed
[model_service-12] [INFO] [1765965975.153285944] [model_service]: Dify LLM connection successful
[model_service-12] [INFO] [1765965975.154461154] [model_service]: ROS_LLM_Service Initialization completed
```

On the virtual machine, create a new terminal and start.

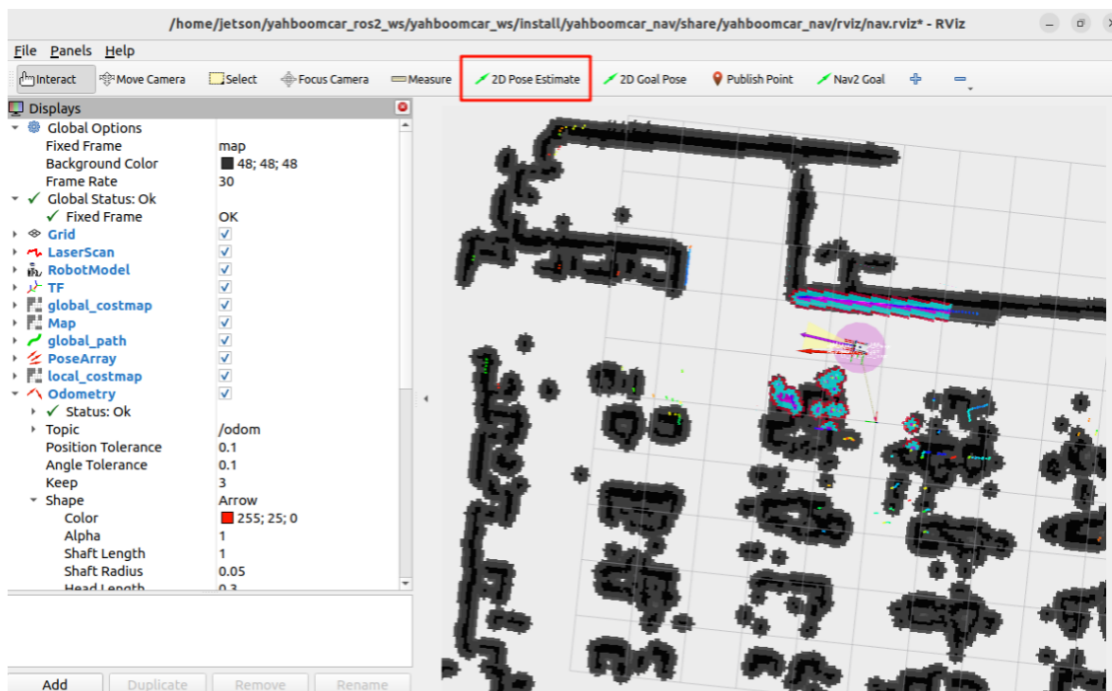
```
ros2 launch yahboomcar_nav display_nav_launch.py
```

Wait for the navigation algorithm to start and the map will be displayed. Then open a virtual machine terminal and enter

```
ros2 launch yahboomcar_nav navigation_teb_launch.py
```

**Note: When running the car's mapping function, you must enter the save map command on the virtual machine to save the navigation map on the virtual machine.**

Then follow the navigation function startup process to initialize positioning. The rviz2 visualization interface will open. Click **2D Pose Estimate** in the upper toolbar to select it. Roughly mark the robot's position and orientation on the map. After initializing positioning, preparations are complete.



### 3.1.1 RDKX5, Raspberry Pi PI5 and ORIN board startup steps:

Open the host terminal and start the Dify environment.

```
sh ~/bringup_dify.sh
```

```
pi@raspberrypi: ~  
File Edit Tabs Help  
pi@raspberrypi:~ $ sh ~/bringup_dify.sh  
[+] Running 13/13  
✓ Network docker_ssrf_proxy_network Created 0.1s  
✓ Network docker_default Created 0.1s  
✓ Container docker-weaviate-1 Started 2.2s  
✓ Container docker-redis-1 Started 2.6s  
✓ Container docker-web-1 Started 3.1s  
✓ Container docker-sandbox-1 Started 2.2s  
✓ Container docker-db-1 Healthy 4.6s  
✓ Container docker-ssrf_proxy-1 Started 3.1s  
✓ Container docker-worker_beat-1 Started 7.3s  
✓ Container docker-worker-1 Started 7.2s  
✓ Container docker-plugin_daemon-1 Started 6.2s  
✓ Container docker-api-1 Started 6.8s  
✓ Container docker-nginx-1 Started 7.7s  
pi@raspberrypi:~ $
```

For Raspberry Pi 5, you need to first enter the Docker container. For RDKX5 and Orin board, this is not necessary.

Open a terminal in Docker and enter the following command:

```
ros2 launch multi_brains llm_agent_control.launch.py text_chat_mode:=True
```

After initialization is complete, the following will be displayed:

```
[usb_cam_node_exe-1] YUYV 4:2:2 800 x 600 (20 Hz)  
[usb_cam_node_exe-1] YUYV 4:2:2 1280 x 1024 (6 Hz)  
[usb_cam_node_exe-1] YUYV 4:2:2 320 x 240 (30 Hz)  
[usb_cam_node_exe-1] [INFO] [1765965894.552487562] [usb_cam]: Setting 'white_balance_temperature_auto' to 1  
[usb_cam_node_exe-1] [INFO] [1765965894.552563191] [usb_cam]: Setting 'exposure_auto' to 1  
[usb_cam_node_exe-1] [INFO] [1765965894.552578635] [usb_cam]: Setting 'exposure' to 150  
[usb_cam_node_exe-1] [INFO] [1765965894.585467996] [usb_cam]: Setting 'focus_auto' to 0  
[usb_cam_node_exe-1] [INFO] [1765965894.673818636] [usb_cam]: Timer triggering every 8 ms  
[INFO] [model_service-12]: process started with pid [439]  
[INFO] [action_service_usb-13]: process started with pid [441]  
[action_service_usb-13] [INFO] [1765965908.031574888] [action_service_usb]: ROS_Action_Service Initialization completed  
[model_service-12] [INFO] [1765965975.153285944] [model_service]: Dify LLM connection successful  
[model_service-12] [INFO] [1765965975.154461154] [model_service]: ROS_LLM_Service Initialization completed
```

Create a new terminal on the virtual machine and start it. (For Raspberry Pi and Jetson Nano, it is recommended to run visualization in the virtual machine)

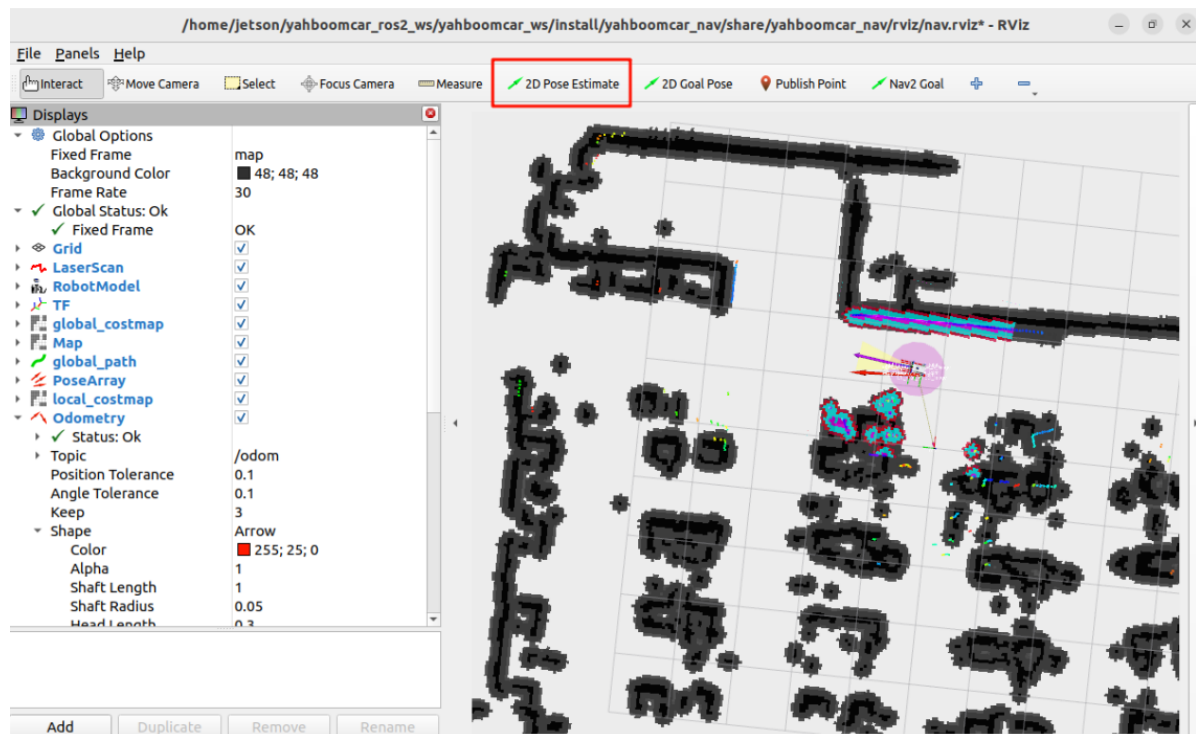
```
ros2 launch yahboomcar_nav display_nav_launch.py
```

Wait for the navigation algorithm to start and the map will be displayed. Then open the Docker terminal and enter

```
#Choose one of the two navigation algorithms  
#Normal navigation  
ros2 launch yahboomcar_nav navigation_teb_launch.py  
  
#Fast Relocation Navigation (Not supported on RDKX5, Raspberry Pi 5, and Jetson Nano main controllers)  
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=true  
load_state_filename:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahboomcar.pbstream  
  
ros2 launch yahboomcar_nav navigation_cartodwb_launch.py  
maps:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahboomcar.yaml
```

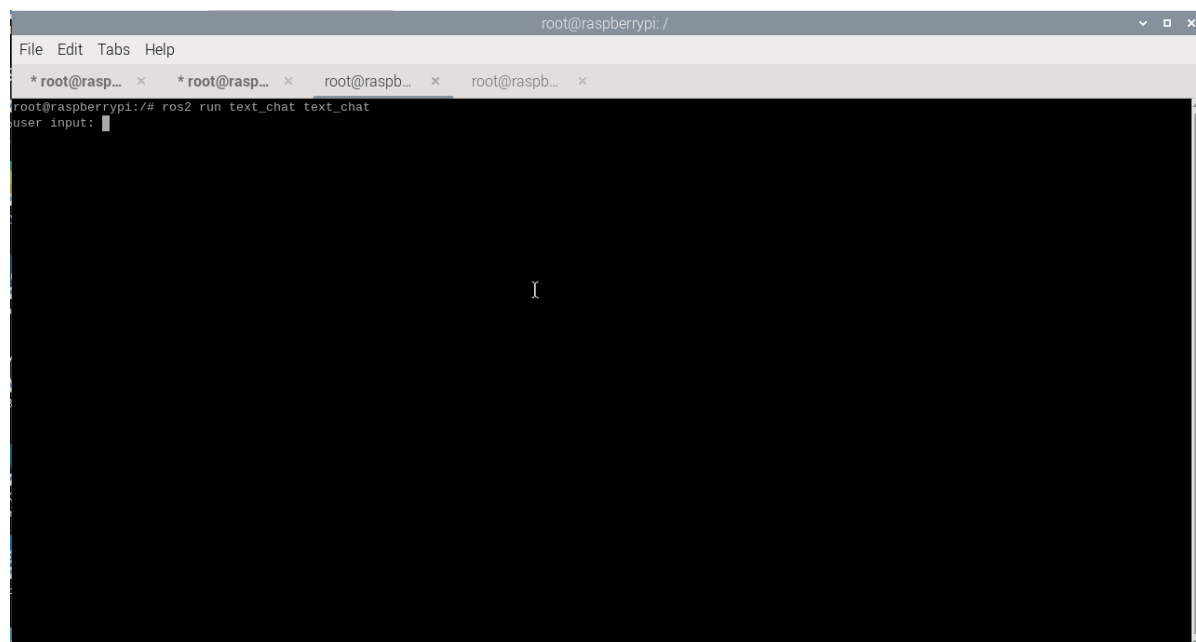
**Note: yahboomcar.yaml and yahboomcar.pbstream must be mapped simultaneously, meaning they are the same map. Refer to the cartograph mapping algorithm to save the map.**

After that, follow the navigation launch process to initialize positioning. The rviz2 visualization interface will open. Click **2D Pose Estimate** in the upper toolbar to select it. Roughly mark the robot's position and orientation on the map. After initial positioning, preparations are complete.



Enter the same docker in multiple terminals and start it.

```
ros2 run text_chat text_chat
```





## 3.2 Test Cases

Here are some reference test cases; users are welcome to create their own dialogue commands.

- Please remember your current location, then navigate to the kitchen and office area, remember what you see, and then return to your starting location. Tell me what you see in each of these two places.

### 3.2.1 Example 1

Type "Please remember your current location, then navigate to the kitchen and office area, remember what you see, and then return to your starting location. Tell me what you see in each of these two places?" in the terminal. The terminal will print the following message:

```
root@raspberrypi: /
File Edit Tabs Help

* root@raspb... × * root@raspb... × root@raspb... × root@raspb... ×

root@raspberrypi:~/# ros2 run text_chat text_chat
user input: 请记住现在的位置，然后依次导航去厨房和办公区记住你所看到的物体，最后返回出发时的位置告诉我你都在那两个地方看到了什么
okay👉, let me think for a moment... |[INFO] [1755676667.184244189] [text_chat_node]: 决策层AI规划:1.调用记录当前位置函数，记录位置
2.导航到厨房
3.调用视觉函数观察周围环境
4.记录厨房中看到的物体
5.导航到办公区
6.调用视觉函数观察周围环境
7.记录办公区中看到的物体
8.调用返回初始位置函数回到出发时的位置
9.反馈在厨房和办公区看到的物体
user input: [INFO] [1755676672.398746540] [text_chat_node]: "action": ['get_current_pose()'], "response": 好的呀，我现在先记下当前位置，就像给我的小地图做个标记一样
|[INFO] [1755676674.254860996] [text_chat_node]: "action": ['navigation(G)'], "response": 出发咯，我正朝着厨房前进，准备开启我的探索之旅
|[INFO] [1755676713.932795368] [text_chat_node]: "action": ['seewhat()'], "response": 我已经到达厨房啦，现在用我的火眼金睛仔细观察一下这里都有什么
|[INFO] [1755676716.382806416] [text_chat_node]: "action": [], "response": 我看到厨房里有一个纸箱、一个黑色的柜子，还有一个人坐在椅子上，看起来像是在休息
|[INFO] [1755676721.428368681] [text_chat_node]: "action": ['navigation(F)'], "response": 现在我准备前往办公区，看看那里又有什么有趣的东西
|[INFO] [1755676797.221251364] [text_chat_node]: "action": ['navigation(F)'], "response": 哎呀，我好像在去办公区的路上遇到一点小麻烦，让我再试一次
|[INFO] [1755676805.926539462] [text_chat_node]: "action": ['seewhat()'], "response": 我已经到达办公区啦，现在开始仔细观察这里都有什么
|[INFO] [1755676809.408347093] [text_chat_node]: "action": [], "response": 我看到办公区有几张桌子，上面放着几瓶水，远处还有一个红色的消防柜，地面很干净，反射出灯光
|[INFO] [1755676814.552218835] [text_chat_node]: "action": ['navigation(0)'], "response": 现在我准备返回出发时的位置，把看到的都告诉您
|[INFO] [1755676839.209056955] [text_chat_node]: "action": [], "response": 我已经回到出发点啦！在厨房我看到了一个纸箱、一个黑色柜子和一个人坐在椅子上；在办公区我看到了几张桌子、几瓶水和一个红色消防柜。全部任务完成，有需要再叫我哦~
|[INFO] [1755676841.390891038] [text_chat_node]: "action": ['finishtask()'], "response": 我已经完成全部任务啦，还有什么需要帮您的吗？
```

The decision-layer model outputs the planned task steps:

```
okay👉, let me think for a moment... |[INFO] [1755676667.184244189] [text_chat_node]: 决策层AI规划:1.调用记录当前位置函数，记录位置
2.导航到厨房
3.调用视觉函数观察周围环境
4.记录厨房中看到的物体
5.导航到办公区
6.调用视觉函数观察周围环境
7.记录办公区中看到的物体
8.调用返回初始位置函数回到出发时的位置
9.反馈在厨房和办公区看到的物体
```

The execution-layer model then executes the task steps:

```
user input: [INFO] [1755676672.398746540] [text_chat_node]: "action": ['get_current_pose()'], "response": 好的呀，我现在先记下当前位置，就像给我的小地图做个标记一样
|[INFO] [1755676674.254860996] [text_chat_node]: "action": ['navigation(G)'], "response": 出发咯，我正朝着厨房前进，准备开启我的探索之旅
|[INFO] [1755676713.932795368] [text_chat_node]: "action": ['seewhat()'], "response": 我已经到达厨房啦，现在用我的火眼金睛仔细观察一下这里都有什么
|[INFO] [1755676716.382806416] [text_chat_node]: "action": [], "response": 我看到厨房里有一个纸箱、一个黑色的柜子，还有一个人坐在椅子上，看起来像是在休息
|[INFO] [1755676721.428368681] [text_chat_node]: "action": ['navigation(F)'], "response": 现在我准备前往办公区，看看那里又有什么有趣的东西
|[INFO] [1755676797.221251364] [text_chat_node]: "action": ['navigation(F)'], "response": 哎呀，我好像在去办公区的路上遇到一点小麻烦，让我再试一次
|[INFO] [1755676805.926539462] [text_chat_node]: "action": ['seewhat()'], "response": 我已经到达办公区啦，现在开始仔细观察这里都有什么
|[INFO] [1755676809.408347093] [text_chat_node]: "action": [], "response": 我看到办公区有几张桌子，上面放着几瓶水，远处还有一个红色的消防柜，地面很干净，反射出灯光
|[INFO] [1755676814.552218835] [text_chat_node]: "action": ['navigation(0)'], "response": 现在我准备返回出发时的位置，把看到的都告诉您
|[INFO] [1755676839.209056955] [text_chat_node]: "action": [], "response": 我已经回到出发点啦！在厨房我看到了一个纸箱、一个黑色柜子和一个人坐在椅子上；在办公区我看到了几张桌子、几瓶水和一个红色消防柜。全部任务完成，有需要再叫我哦~
|[INFO] [1755676841.390891038] [text_chat_node]: "action": ['finishtask()'], "response": 我已经完成全部任务啦，还有什么需要帮您的吗？
```

After completing the task, the robot enters a waiting state. Instructions from this state are directly passed to the execution-layer model, and all conversation history is retained. You can press the **ENTER** key in the terminal to continue the conversation and enter the "End current task" command to terminate the current task cycle and start a new one.



```
root@raspberrypi: /
File Edit Tabs Help

*root@rasp... x *root@rasp... x root@rasp... x root@rasp... x

root@raspberrypi: /# ros2 run text_chat text_chat
user input: 请记住现在的位置，然后依次导航去厨房和办公区记住你所看到的物体，最后返回出发时的位置告诉我都在那两个地方看到了什么
okay☺, let me think for a moment... [INFO] [1755676667.184244189] [text_chat_node]: 决策层AI规划:1.调用记录当前位置函数，记录位置
2.导航到厨房
3.调用视觉函数观察周围环境
4.记录厨房中看到的物体
5.导航到办公区
6.调用视觉函数观察周围环境
7.记录办公区中看到的物体
8.调用返回初始位置函数回到出发时的位置
9.反馈在厨房和办公区看到的物体
user input: [INFO] [1755676672.398746540] [text_chat_node]: "action": ['get_current_pose()'], "response": 好的呀，我现在先记下当前位置，就像给我的小地图做个标记一样
[INFO] [1755676674.254860996] [text_chat_node]: "action": ['navigation(G)'], "response": 出发咯，我正朝着厨房前进，准备开启我的探索之旅
[INFO] [1755676713.932795368] [text_chat_node]: "action": ['seewhat()'], "response": 我已经到达厨房啦，现在用我的火眼金睛仔细观察一下这里都有什么
[INFO] [1755676716.382806416] [text_chat_node]: "action": [], "response": 我看到厨房有一个纸箱、一个黑色的柜子，还有一个人坐在椅子上，看起来像是在休息
[INFO] [1755676721.428368681] [text_chat_node]: "action": ['navigation(F)'], "response": 现在我准备前往办公区，看看那里又有什么有趣的东西
[INFO] [1755676797.221251364] [text_chat_node]: "action": ['navigation(F)'], "response": 哎呀，我好像在去办公区的路上遇到一点小麻烦，让我再试一次
[INFO] [1755676805.926539462] [text_chat_node]: "action": ['seewhat()'], "response": 我已经到达办公区啦，现在开始仔细观察这里都有什么
[INFO] [1755676809.408347093] [text_chat_node]: "action": [], "response": 我看到办公区有几张桌子，上面放着几瓶水，远处还有一个红色的消防柜，地面很干净，反射出灯光
[INFO] [1755676814.552218835] [text_chat_node]: "action": ['navigation(zero)'], "response": 现在我准备返回出发时的位置，把看到的東西都告诉您
[INFO] [1755676839.209056955] [text_chat_node]: "action": [], "response": 我已经回到出发点啦！在厨房我看到了一个纸箱、一个黑色柜子和一个人坐在椅子上；在办公区我看到了几张桌子、几瓶水和一个红色消防柜。全部任务完成，有需要再叫我哦~
[INFO] [1755676841.390891038] [text_chat_node]: "action": ['finishtask()'], "response": 我已经完成全部任务啦，还有什么需要帮您的吗？
user input: 结束当前任务
okay☺, let me think for a moment... [INFO] [1755677530.563305659] [text_chat_node]: "action": ['finish_dialogue()'], "response": 好的，任务已经结束了，有需要再叫我哦~
user input: █
```

## 4. Source Code Parsing

Source code located at:

Jetson Orin Nano:

```
#NUWA Camera User
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_nuwa.py
#USB Camera User
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_usb.py
```

Jetson Nano, Raspberry Pi :

You need to first enter Docker.

```
#NUWA Camera User
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_nuwa.py
#USB Camera User
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_usb.py
```

RDKX5:

```
#NUWA Camera User
/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_nuwa.py
#USB Camera User
/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_usb.py
```

## 4.1 Example 1

action\_service.py:

Example 1 uses the **seewhat**, **navigation**, **load\_target\_points**, and **get\_current\_pose** methods in the **CustomActionServer** class. **seewhat** was already explained in the **Multimodal Visual Understanding** section. This section explains the newly introduced **navigation**, **load\_target\_points**, and **get\_current\_pose** functions.

In the **init\_ros\_communication** initialization function, a nav2 navigation client is created to request the ROS2 navigation action server for subsequent navigation target point requests. A TF listener is created to monitor coordinate transformations between the map and basefootprint.

```
#Create the navigation client and request the navigation action server
self.navclient = ActionClient(self, NavigateToPose, 'navigate_to_pose')
#Create the TF listener to monitor coordinate transformations
self.tf_buffer = Buffer()
self.tf_listener = TransformListener(self.tf_buffer, self)
```

The **load\_target\_points** function loads the target point coordinates from the map mapping file, map\_mapping.yaml, and creates a navigation dictionary to store characters and their corresponding map coordinates. Each point coordinate is a PoseStamped data type.

```
def load_target_points(self):
    with open(self.map_mapping_config, 'r') as file:
        target_points = yaml.safe_load(file)
    self.navpose_dict = {}
    for name, data in target_points.items():
        pose = PoseStamped()
        pose.header.frame_id = 'map'
        pose.pose.position.x = data['position']['x']
        pose.pose.position.y = data['position']['y']
        pose.pose.orientation.x = data['orientation']['x']
        pose.pose.orientation.y = data['orientation']['y']
        pose.pose.orientation.z = data['orientation']['z']
        pose.pose.orientation.w = data['orientation']['w']
        self.navpose_dict[name] = pose
```

**navigation** function: receives a character parameter (corresponding to the character in the above map mapping), parses the coordinates of the corresponding character from the dictionary, and uses the self.navclient navigation client object to request the navigation action server of ros2. When the navigation action server returns a value of 4, it means that the navigation is successful, and other values represent failures (possibly due to obstacles, planning failures, etc.). self.nav\_status indicates that the navigation flag can be interrupted through dialogue, and the result of the action execution is fed back to the large model after the navigation is completed.

```
def navigation(self, point_name):
    """
    Get the target point coordinates from the navpose_dict dictionary and
    navigate to the target point
    """
    self.nav_runing = True
    point_name = point_name.strip("\")
    if point_name not in self.navpose_dict:
        self.get_logger().error(
```

```

        f"Target point '{point_name}' does not exist in the navigation
dictionary."
    )
    self.action_status_pub("navigation_3", point_name=point_name)
    return

if self.first_record:
    # 出发前记录当前在全局地图中的坐标(只有在每个任务周期的第一次执行时才会记录)/ before
starting a new task, record the current pose in the global map
    transform = self.tf_buffer.lookup_transform(
        "map", "base_footprint", rclpy.time.Time()
    )
    pose = PoseStamped()
    pose.header.frame_id = "map"
    pose.pose.position.x = transform.transform.translation.x
    pose.pose.position.y = transform.transform.translation.y
    pose.pose.position.z = 0.0
    pose.pose.orientation = transform.transform.rotation
    self.navpose_dict["zero"] = pose
    self.first_record = False

# Get the target point coordinates
target_pose = self.navpose_dict.get(point_name)
goal_msg = NavigateToPose.Goal()
goal_msg.pose = target_pose
send_goal_future = self.navclient.send_goal_async(goal_msg)

def goal_response_callback(future):
    goal_handle = future.result()
    if not goal_handle or not goal_handle.accepted:
        self.get_logger().error("Goal was rejected!")
        self.action_status_pub("navigation_1", point_name=point_name)
        return

    get_result_future = goal_handle.get_result_async()

    def result_callback(future_result):
        result = future_result.result()

        if self.nav_status: #Interrupt sign
            self.nav_status = False
            self.action_status_pub("navigation_5", point_name=point_name)
            self.nav_runing = False

        else:
            if result.status == 4:
                self.action_status_pub(
                    "navigation_2", point_name=point_name
                )# 执行导航成功 /execute navigation success
                self.nav_runing = False
            else:
                self.get_logger().info(
                    f"Navigation failed with status: {result.status}"
                )
                self.action_status_pub(
                    "navigation_4", point_name=point_name
                )# 执行导航失败 /execute_navigation_failed
                self.nav_runing = False

```

```
get_result_future.add_done_callback(result_callback)

send_goal_future.add_done_callback(goal_response_callback)
```

The **get\_current\_pose** function is used to record the robot's current map coordinates in the global coordinate system map and put the coordinates into a dictionary for subsequent retrieval.

```
def get_current_pose(self):#Record the current coordinates in record_pose
    """
    Get the current position in the global map coordinate system
    """
    # Get the current target point coordinates
    transform = self.tf_buffer.lookup_transform(
        'map',
        'base_footprint',
        rclpy.time.Time())
    # Extracting position and pose
    pose = PoseStamped()
    pose.header.frame_id = 'map'
    pose.pose.position.x = transform.transform.translation.x
    pose.pose.position.y = transform.transform.translation.y
    pose.pose.position.z = 0.0
    pose.pose.orientation= transform.transform.rotation
    self.navpose_dict['zero'] = pose
    # Print recorded coordinates
    position = pose.pose.position
    orientation = pose.pose.orientation
    self.get_logger().info(f'Recorded Pose - Position: x={position.x}, y=
    {position.y}, z={position.z}')
    self.get_logger().info(f'Recorded Pose - Orientation: x={orientation.x}, y=
    {orientation.y}, z={orientation.z}, w={orientation.w}')
    self.action_status_pub(f'机器人反馈:get_current_pose()成功')
```

